

WORLDLINE GLOBAL

Round 2 — SDET / Test Automation Lead

Master Interview Revision & Practice Notes

Focus	Java • Debugging • Defensive Coding • API • Mocking • Concurrency • Automation
Current coverage	Exception handling, concurrency, validation, money precision, debugging
Practice format	Question → Your answer → Correction → Simple explanation → Interview-ready answer
Next coverage	Test design → API/mocking → JUnit/Mockito → Spring Boot → reporting → framework → leadership

Living document: We will continue adding the one-by-one interview questions, corrections, examples and polished answers to this revision structure.

1. Worldline Round 2 — What the Feedback Tells Us

Rejected-candidate gaps: defensive behaviour; restricted answers; API testing/mock; third-party server unavailable; debugging and exception handling; Exception vs Error; NPE; concurrency/race conditions; floating-point precision; Allure/Extent reporting; Spring Boot/backend; PUT/DELETE idempotency; lack of curiosity.

Selected-candidate strengths: strong coding logic; structured problem solving; finding multiple bugs; safe concurrency using synchronized; defensive null/0/negative validation; comprehensive positive/negative/boundary tests; JUnit + Mockito for external dependencies; methodological engineering approach; hands-on debugging.

Preparation principle: For each topic, explain the reason, demonstrate a practical example, identify edge cases, and connect the answer to reliable automation and debugging.

2. Java Coding, Debugging & Defensive Coding

Original code reviewed

```
public class OrderService {
    private double total = 0;

    public void addItem(Double price, Integer quantity) {
        if (quantity > 0) {
            total = total + price * quantity;
        }
    }

    public double getTotal() { return total; }
}
```

Problems identified: nullable wrappers, missing validation, zero/negative values, floating-point money calculation, shared mutable state and concurrency risk.

Q1 — Why are Double and Integer risky?

Explanation: They are nullable wrapper objects. Arithmetic triggers auto-unboxing; if either is null, Java can throw NullPointerException.

How I would handle it: Validate before arithmetic; fail fast with a meaningful IllegalArgumentException.

Interview-ready: “I validate nullable inputs before unboxing. Validation is better than catching NPE because it gives a clear contract violation and fails fast.”

Q2 — quantity = 0

Explanation: If an order line must contain at least one item, zero is invalid and should be rejected.

How I would handle it: Use quantity <= 0 when the business rule is positive quantity.

Interview-ready: “I validate the business rule explicitly and reject zero if an order line requires at least one item.”

Q3 — negative quantity

Explanation: Normally invalid for a purchase line, unless the domain explicitly models returns or adjustments.

How I would handle it: Reject with quantity <= 0 for normal purchases.

Interview-ready: “I would reject negative quantity in the normal purchase flow and model returns/adjustments separately if required.”

Q4 — price = 0

Explanation: Zero can be valid for free/promotional items, so confirm the business rule instead of automatically rejecting it.

How I would handle it: Use price < 0 when only negative prices are invalid.

Interview-ready: “Zero price is not automatically invalid; I would confirm the business rule because promotional or free items may legitimately cost zero.”

Q5 — negative price

Explanation: Normally invalid for a standard purchase unless the domain uses negative adjustments.

How I would handle it: Reject negative price when the business contract requires non-negative prices.

Interview-ready: “I would reject negative prices unless the domain explicitly supports credits or adjustments.”

Q6 — double for money

Explanation: Binary floating point cannot represent many decimal fractions exactly; $0.1 + 0.2$ can produce 0.30000000000000004 .

How I would handle it: Use `BigDecimal`. Prefer `BigDecimal.valueOf(...)` or a `String` constructor such as `new BigDecimal("10.50")`.

Interview-ready: “For money I use `BigDecimal` because financial calculations require predictable decimal precision.”

Q7 — concurrency problem

Explanation: `total` is shared mutable state. `total = total + amount` is a read-modify-write sequence, so concurrent updates can be lost.

How I would handle it: Protect the critical section with `synchronized`, or use an atomic type for simple counters.

Interview-ready: “The update is not atomic. Two threads can read the same old value and overwrite each other. `synchronized` protects the critical section for the same object.”

Q8 — `AtomicInteger`

Explanation: Useful for simple shared integer counters because operations such as `incrementAndGet()` are atomic.

How I would handle it: Use `AtomicInteger` for simple shared counters; use `synchronized`/locks for larger related critical sections.

Interview-ready: “`AtomicInteger` is appropriate for simple shared counters; it is not a universal replacement for `synchronized`.”

Q9 — `ThreadLocal`

Explanation: `ThreadLocal` gives each thread its own independent value. It does not solve a requirement for one shared counter.

How I would handle it: Good example: one `WebDriver` instance per parallel test thread.

Interview-ready: “`ThreadLocal` is for isolated per-thread state; `AtomicInteger` is for shared integer state requiring atomic updates.”

3. Exception Handling — Completed Practice

Q10 — Exception vs Error

Explanation: Exception represents application/runtime conditions that may be handled. Error generally represents serious JVM/system-level problems such as `OutOfMemoryError` or `StackOverflowError`. Syntax errors are compiler errors, not `java.lang.Error`.

Interview-ready: “I distinguish application exceptions from serious JVM errors. I generally handle appropriate exceptions, but I do not broadly catch `Error`.”

Q11 — Is `NullPointerException` checked?

Explanation: No. `NPE` extends `RuntimeException`, so it is unchecked and the compiler does not force handling.

Interview-ready: “`NullPointerException` is an unchecked `RuntimeException`; the compiler does not force handling at compile time.”

Q12 — Checked vs unchecked

Explanation: Checked exceptions must be handled or declared. Unchecked exceptions extend `RuntimeException` and are not compiler-enforced.

Interview-ready: “Checked exceptions represent conditions the application may reasonably handle, such as I/O failures. Unchecked exceptions commonly represent programming, validation or runtime problems.”

Q13 — throw vs throws

Explanation: `throw` actually raises an exception. `throws` declares that a method may propagate an exception.

Interview-ready: “`throw` is the action of raising an exception; `throws` is part of the method contract declaring possible propagation.”

Q14 — finally

Explanation: `finally` normally executes whether an exception occurs or not and is useful for cleanup/actions that should normally happen regardless of success or failure.

Interview-ready: “`finally` is useful for general cleanup. For `AutoCloseable` resources, I prefer `try-with-resources`.”

Q15 — try-with-resources

Explanation: Java automatically closes `AutoCloseable` resources when the `try` block exits, including when an exception occurs.

Interview-ready: “For files and JDBC resources, `try-with-resources` is safer and cleaner than manual closing in `finally`.”

Q16 — catch(Exception e)

Explanation: It is broad and may catch unrelated exceptions. If handled poorly, it can hide unexpected product defects and reduce diagnostic quality.

Interview-ready: “I catch the most specific exception that I can meaningfully handle.”

Q17 — swallowing an exception

Explanation: Catching and doing nothing hides the failure and can allow automation to continue in an invalid state.

Interview-ready: “This is exception swallowing. It is dangerous because the original failure and stack trace are lost, and later failures may be misleading.”

Q18 — exception propagation

Explanation: If not handled, an exception moves upward through the call stack until a matching catch is found. If nobody handles it, the caller/test framework receives the failure.

Interview-ready: “An unhandled exception propagates up the call stack. If no handler exists, it eventually reaches the caller or test framework and causes failure.”

Q19 — log and rethrow

Explanation: Logging gives diagnostics; rethrowing ensures the failure is not hidden and can be handled by a higher layer/test framework.

Interview-ready: “I log and rethrow when the current layer can add useful context but cannot recover.”

Q20 — throw e vs new RuntimeException(e)

Explanation: throw e rethrows the same exception. new RuntimeException(e) creates a new exception and stores the original as its cause.

Interview-ready: “Rethrow preserves the original exception; wrapping creates a new exception with the original cause and is useful for adding context or translating layers.”

Q21 — when to wrap

Explanation: Wrap when a layer needs to add business/context meaning or translate a low-level exception into an application-specific exception, while retaining the original cause.

Interview-ready: “I wrap an exception when I need to add meaningful context while preserving the root cause for debugging.”

Q22 — one large try-catch around the whole test

Explanation: It can hide the exact failure point, swallow diagnostics and make debugging difficult. Unexpected test failures should generally reach the test framework.

Interview-ready: “I avoid a generic catch around the whole test. I use targeted handling only when I can add value; otherwise I let the framework capture the failure and stack trace.”

4. Rapid Revision — Must Remember

Topic	One-line memory
Wrapper + null	Auto-unboxing a null wrapper can cause NPE.
Validation	Validate input before business logic; fail fast with meaningful messages.
Money	Use BigDecimal, not double.
Race condition	Shared mutable state + non-atomic update can cause lost updates.
synchronized	One thread at a time for the same lock/object.
AtomicInteger	Atomic operations for simple shared integer state.
ThreadLocal	Separate value per thread; not a solution for shared counters.
Exception	Application/runtime condition that may be handled.
Error	Serious JVM/system-level problem; don't broadly catch it.
Checked	Compiler forces handle or declare.
Unchecked	RuntimeException family; compiler does not force handling.
throw	Actually raises an exception.
throws	Declares possible propagation.
finally	Normally executes regardless of exception.
try-with-resources	Automatically closes AutoCloseable resources.
Swallowing	Catch + ignore = hidden failure.
Propagation	Unhandled exception moves upward through call stack.
Rethrow	Same exception passed upward.
Wrap	New exception with original as cause; adds context.

5. Upcoming Worldline Practice Roadmap

Area	Planned coverage
Test-case design	Concurrency, race conditions, boundary testing, negative cases, floating-point precision, prioritisation and comp
API testing & mocking	REST fundamentals, PUT/PATCH/POST, DELETE, idempotency, third-party unavailable, contract validation, retr
JUnit + Mockito	Mocks, spies, stubs, verify, ArgumentCaptor, external dependency isolation, unit vs integration tests.
Concurrency deeper dive	synchronized, locks, atomic classes, volatile, ExecutorService, thread safety in parallel tests.
Spring Boot / backend	Controller-Service-Repository, dependency injection, REST endpoints, validation, exception handling and testabi
Allure / reporting	Artifacts, screenshots, logs, attachments, listeners, CI integration and failure diagnostics.
Selenium → Playwright	Migration rationale, auto-waiting, locator strategy, parallelism, fixtures, API+UI, visual/accessibility testing.
AI-assisted automation	Cursor/Claude with repository rules and skills, safe usage, review/validation and productivity without ad-hoc prom
Framework & CI/CD	POM, reusable utilities, Jenkins, Git/Bitbucket, Docker/Kubernetes, quality gates and flaky-test control.
Leadership & curiosity	SDET Lead ownership, technical decisions, mentoring, shift-left, risk-based testing and asking strong questions.

Future update rule: Every new practice question will be added with your answer, correction, simple explanation, practical example, common trap and polished interview-ready response.